

Concurrency & Distributed Systems

Week 1 — Part 5: Case Studies: Event Loops, Locks, Queues, and Contention

Dr. Mohamad Aoude

Lebanese University
Faculty of Engineering

May 11, 2026

From Threads to Architecture

We move from:

- “Add threads”
- to
- “Choose the right concurrency model”

Four case studies:

- Single-thread event loop (Redis style)
- Lock-based shared state (contention)
- Queue growth (Little’s Law)
- Read/write contention (SQLite model)

Program 1: Redis-Style Event Loop

Idea:

- One thread processes requests in a loop
- No shared-state races in request handling
- No locks around the “work”
- Works well when per-request work is tiny (in-memory)

Goal

Show that single-threaded can be *fast* and *predictable*.

Core Idea (Simplified)

```
// Single-threaded event loop (Redis-style)
while (running) {
    Request r = queue.poll(); // take next request
    if (r != null) {
        int x = 1 + 1; // tiny in-memory work
        // respond
    }
}
```

Observation

No locks in the processing path. Predictable latency until overload.

How to Run (Program 1)

Compile & run:

```
javac SingleThreadEventLoop.java  
java SingleThreadEventLoop
```

What to observe:

- high throughput
- stable latency when tasks are tiny
- failure mode is queue growth (not races)

Program 2: Shared State + Lock Contention

Idea:

- Many threads update one shared counter
- Correctness requires a lock
- Lock serializes the critical section

Hypothesis to test

“More threads should make it faster.”

Critical Section

```
lock.lock();  
try {  
    counter++; // shared mutable state  
} finally {  
    lock.unlock();  
}
```

Key point

All threads line up here. This becomes a bottleneck.

How to Run (Program 2)

Compile:

```
javac ThreadedCounterWithLock.java
```

How to Run (Program 2)

Compile:

```
javac ThreadedCounterWithLock.java
```

Run with increasing thread counts:

```
java ThreadedCounterWithLock 1  
java ThreadedCounterWithLock 2  
java ThreadedCounterWithLock 4  
java ThreadedCounterWithLock 8  
java ThreadedCounterWithLock 16
```

Expected:

- throughput rises a bit
- then plateaus
- then degrades (contention + context switching)

Program 3: Little's Law and Queue Growth

Little's Law:

$$L = \lambda W$$

- L : average number of requests inside the system
- λ : arrival rate (requests/sec)
- W : time in system (service + waiting)

Interpretation

If λ rises (or W rises), queues grow.

How to Run (Program 3)

Compile & run:

```
javac QueueingLittleLawDemo.java  
java QueueingLittleLawDemo
```

Experiment: overload it

- Open QueueingLittleLawDemo.java
- Increase arrivalRate (e.g., 50 $\rightarrow$ 200)
- Run again and observe Remaining in queue

What Students Must Conclude (Program 3)

- If service time stays the same and arrivals increase, the queue must grow.
- Event loops (Redis style) **must keep work per request tiny**.
- Blocking I/O inside the event loop increases W dramatically.

Bridge

This is why Redis avoids blocking calls in the main loop.

Program 4: SQLite Model (Many Readers, One Writer)

SQLite-like principle:

- Many readers can proceed together
- Writers serialize (exclusive access)

We simulate this with `ReentrantReadWriteLock`:

- read lock for readers
- write lock for writers

Read/Write Lock Pattern

```
lock.readLock().lock();
try {
    // read shared state
} finally {
    lock.readLock().unlock();
}

lock.writeLock().lock();
try {
    // write shared state
} finally {
    lock.writeLock().unlock();
}
```

Observation

Writers block everyone. Contention creates latency spikes.

How to Run (Program 4)

Compile & run:

```
javac SQLiteConcurrencyDemo.java  
java SQLiteConcurrencyDemo
```

In class:

- Increase number of writer threads in the code
- Observe how the system becomes “spiky” (more blocking)

Architectural Tradeoffs (The Point of the Case Study)

- Single-thread event loop:
 - simple, predictable
 - fast for tiny in-memory tasks
 - fails by queue growth under overload
- Lock-heavy shared state:
 - correctness cost (locks)
 - contention and poor scaling
 - higher variance / tail latency

Key insight

Engineering is tradeoffs. Single-threaded is a choice, not a limitation.

Student Questions (Submit as Short Answers)

Answer in 3–5 lines each:

- 1 Why is Redis single-threaded for request processing?
- 2 When does a single-thread event loop fail?
- 3 Why did throughput plateau in the lock-based counter?
- 4 Use Little's Law: if W doubles and λ stays constant, what happens to L ?
- 5 Why does SQLite serialize writes?

Next Topic: Event-Driven I/O and Async Architectures